

Plant Disease Classification Task

Plant disease image classifier covers 39 disease categories across multiple plant species. This work builds a classification system from scratch, without relying on any pre-existing plant-disease-specific models. All GPU training was performed on Google Colab using free-tier (T4) resources.

Dataset Structure

The dataset is organized into train and validation splits, each containing one subfolder per plant-disease combination. A companion CSV file (`plant_diseases.csv`) was written by me manually provides the authoritative mapping from raw folder names to structured fields: `plant_name` and `disease_name`. This decoupling of filesystem naming from label semantics prevents accidental label drift when folders are renamed or reorganized.

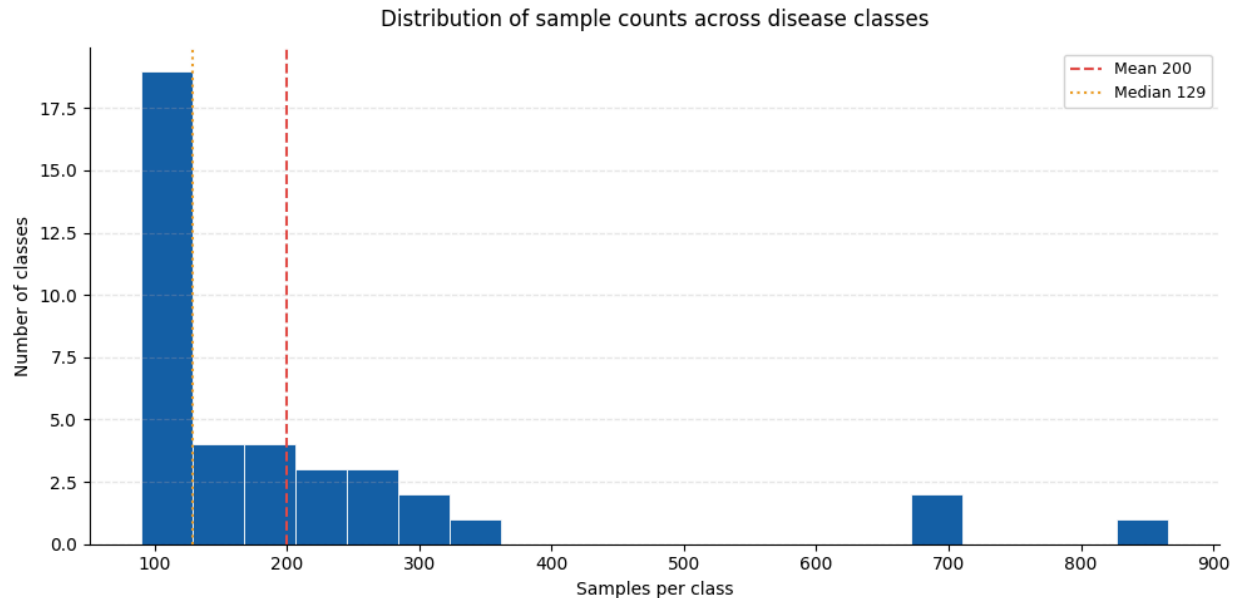
A custom `PlantDiseaseParser` class validates the dataset before any training data is consumed, enforcing three invariants:

1. every folder has a CSV entry
2. no CSV rows reference missing folders
3. no plant/disease fields are blank.

These checks caught several silent mismatches during dataset construction.

Class imbalance

A histogram of per-class training sample counts revealed a long-tailed distribution. Two complementary strategies address this. First, `WeightedRandomSampler` assigns each training image a sampling probability inversely proportional to its class frequency, producing approximately uniform class representation per epoch. Second, label smoothing ($\epsilon = 0.1$) prevents the model from becoming overconfident on majority classes at the expense of minority ones.



Architecture

All models are built the same way. I take a pretrained backbone from the timm library - essentially a network that already knows how to recognize general visual features from training on ImageNet - and strip off its original output layer, since that layer was designed to predict 1000 ImageNet categories, not plant diseases.

On top of the stripped backbone I attach a small custom output block with two components:

- A dropout layer (30% dropout rate) that randomly disables neurons during training. This sits before the final prediction layer and acts as a brake against the model memorizing training images rather than learning generalizable patterns. It's especially useful during Phase 1 when the backbone is frozen and only this output block is being trained.
- A single linear layer that takes the backbone's output and maps it to 39 numbers - one score per disease class.

I split training into two phases to avoid a common pitfall called catastrophic forgetting - where a pretrained network loses its useful ImageNet knowledge the moment you start training it on a new task.

Phase 1 - frozen backbone (10 epochs)

In the first phase I keep the entire backbone frozen, meaning its weights don't change at all. Only the small output head I added gets trained. This lets the model figure out how to map the backbone's existing features to plant disease labels, without touching the backbone itself. I use AdamW as the optimizer with a learning rate of 1e-3, cosine annealing over the 10 epochs, and early stopping that saves the best checkpoint based on validation mAP.

Phase 2 - full fine-tune (up to 50 epochs)

Once the head has settled, I unfreeze the backbone and train everything together. The key detail here is that I use different learning rates for the two parts: $1e-4$ for the head and a much smaller $1e-5$ for the backbone. The backbone gets a 10 times lower rate because I don't want to aggressively overwrite the low-level texture and shape features it learned from ImageNet - I just want to gently nudge them toward plant disease imagery. Early stopping continues with the same patience of 10 epochs.

Augmentation Pipeline

Since I'm working with a limited number of images per class, and many diseases look visually similar to each other, I applied a heavy augmentation pipeline during training. The idea is to artificially increase the variety of what the model sees, forcing it to learn disease-specific features rather than memorizing individual photos.

Used RandomResizedCrop, RandomHorizontalFlip, RandomHorizontalFlip, ColorJitter, RandomRotation, RandAugment. MixUp and CutMix - are only active during training and are applied randomly per batch with equal probability. Instead of feeding the model a clean image with a single correct label, they present a blend of two images and ask the model to predict both labels proportionally. This stops the model from being overconfident and pushes it toward producing more calibrated probability scores, which directly helps mAP.

Evaluation

The primary metric is mean average precision (mAP) - the average of the per-class AP scores across all 39 disease classes. It weights every class equally regardless of how many samples it has. A model that scores perfectly on common diseases but fails on rare ones will still be penalized heavily, which is exactly the behaviour I want for a real-world plant disease tool where missing any disease matters.

Model Selection

I tried four different models before settling on EfficientNet. Table shows the results - the ResNet experiments were stopped early once it became clear they were overfitting.

Model	Phase 1 mAP	Phase 2 mAP	Status
EfficientNet-B0	0.6001	0.7840 (TTA: 0.7868)	Complete
EfficientNet-B2	0.6323	0.8288 (TTA: 0.8347)	Complete
ResNet-18	~0.51	Overfit	Stopped

ResNet-50	~0.44	Overfit	Stopped
------------------	-------	---------	---------

Why ResNets overfit

Both ResNet variants showed the same problem: training loss kept dropping while validation mAP stopped improving within 15 epochs and then started getting worse. There were 3 reasons for this.

The first was simply too many parameters. ResNet-50 has more parameters than EfficientNet-B0 - but my dataset only has a few hundred images per class. That much capacity doesn't learn better features, it just memorizes the training photos. The second was that ResNets have no built-in regularization. A standard ResNet convolution can pick up on tiny texture details that are specific to individual training images but meaningless for generalization. EfficientNet's depthwise separable convolutions are more constrained by design, so they're physically less capable of that kind of memorization.

The third was that ResNet's pretrained weights were simply less robust. ResNet was pretrained on ImageNet with a basic pipeline - no augmentation policies, no dropout. EfficientNet's pretraining was more regularized from the start, so when I fine-tune it on plant disease images it adapts more gracefully instead of immediately locking onto training-set patterns. I tried to fix ResNet-18 by pushing dropout up to 0.5 and weight decay to $5e-4$. It helped slightly but never enough - it still ended up weaker than EfficientNet-B0.

Training Dynamics

EfficientNet-B0 started Phase 1 at mAP 0.456 and steadily climbed to 0.600 by epoch 8, then stalled - which is expected, since the backbone was frozen and the head had learned about as much as it could from static features. Phase 2 improved consistently for the first 30 epochs, then slowed down to small incremental gains, finishing at a best mAP of 0.7840. With TTA that rose to 0.7868. The curve hadn't fully flattened by epoch 50, so a few more epochs would likely have squeezed out a small additional gain.

EfficientNet-B2 showed a noticeably stronger start. Phase 1 already reached 0.6323 - 3.2 points above B0's Phase 1 best - which I attribute to the larger backbone and higher input resolution (260 px vs 224 px giving the model more fine-grained detail per image). Phase 2 progressed steadily, crossing 0.80 mAP at epoch 20 and peaking at 0.8288 by epoch 43. TTA pushed that to 0.8347. The improvement slowed after epoch 37 and early stopping triggered after epoch 43 - the model had genuinely converged.

Comparing the two, B2 finished roughly 4.5 points ahead of B0 (0.8288 vs 0.7840), consistent with the gap that was already visible after Phase 1. The bigger model didn't overfit despite having nearly twice the parameters - the augmentation pipeline, label smoothing, and weighted sampling kept it well regularized throughout.

